Title: DOM XSS via Unsanitized Search Input Allowing Arbitrary Script Execution

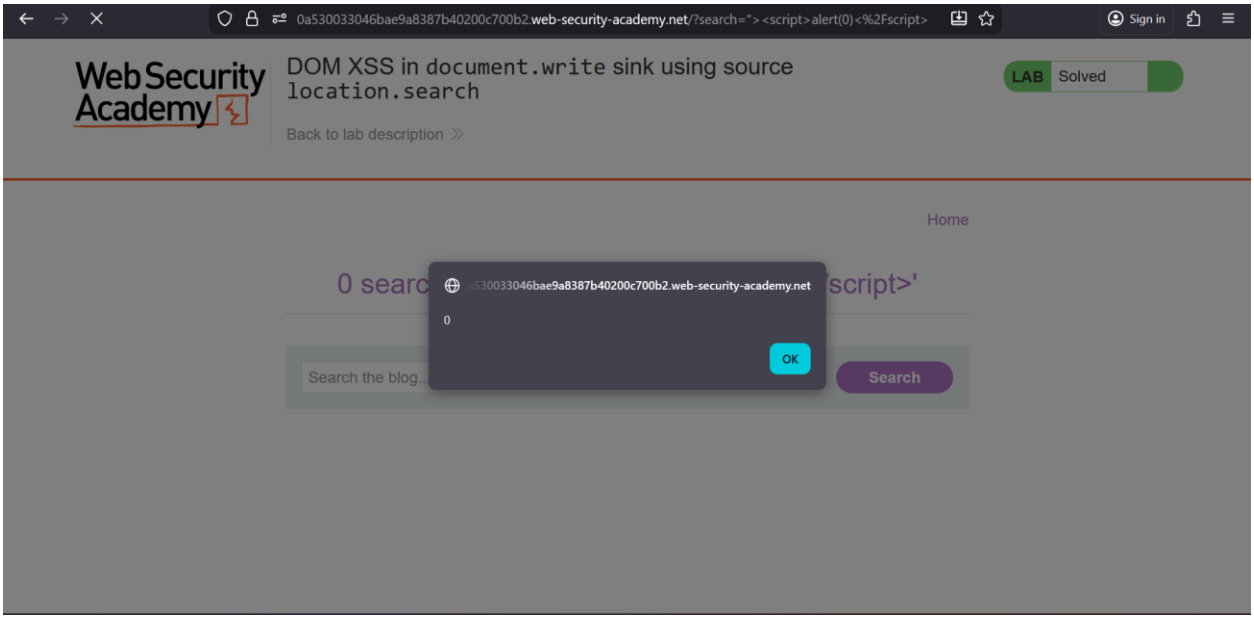
Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user. Flaws that allow these attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates without validating or encoding it.

An attacker can use XSS to send a malicious script to an unsuspecting user. The end user's browser has no way to know that the script should not be trusted, and will execute the script. Because it thinks the script came from a trusted source, the malicious script can access any cookies, session tokens, or other sensitive information retained by the browser and used with that site. These scripts can even rewrite the content of the HTML page

Reproduction steps:

1-Visit: https://0ad600fd0411964a805f0328000d00a9.web-security-academy.net

2-Enter in the comment section the malicious code "><script>alert(0)</script>-



3-now the alert work on the webpage

Impact statement:

Inject and execute arbitrary JavaScript in the victim's browser by simply tricking them into clicking a crafted link
The malicious script runs in the context of the vulnerable site, meaning it has full access to anything that site can access
Steal session cookies and hijack authenticated user sessions instantly.

Mitigation:

1. Encode All User Input Before Rendering in HTML Any value reflected back into the page must be HTML-encoded before output. Characters like `<`, `>`, `"`, `'` must be converted to their safe HTML entity equivalents so the browser treats them as text, not code.

2. Implement a Strong Content Security Policy (CSP) Add a `Content-Security-Policy` header that restricts which scripts are allowed to execute. A strict CSP prevents injected scripts from running even if encoding is missed somewhere.

3. Use a Proven Templating Engine with Auto-Escaping Modern frameworks like React, Angular, and Django templates auto-escape output by default. Never manually build HTML by concatenating raw user input — always go through the framework's safe rendering layer.

4. Validate and Reject Unexpected Input Server-Side A search box should only accept plain text. Reject or strip input containing HTML tags, script keywords, and event handler attributes (`onerror`, `onload`, etc.) before processing.

5. Set the HttpOnly Flag on Session Cookies While this does not prevent XSS itself, marking session cookies as `HttpOnly` ensures injected scripts cannot read them via `document.cookie`, blocking the most common XSS-to-account-takeover path.

6. Never Trust the Referrer or URL Parameters Any value that comes from the URL — query strings, path segments, fragment identifiers — must be treated as untrusted input and encoded before being written into the page response.